

올바른 소통을 통해 팀 프로젝트의 시너지를 극대화 하는

CRA양성과정: 팀커뮤니케이션

K N O W - W H Y & K N O W - H O W



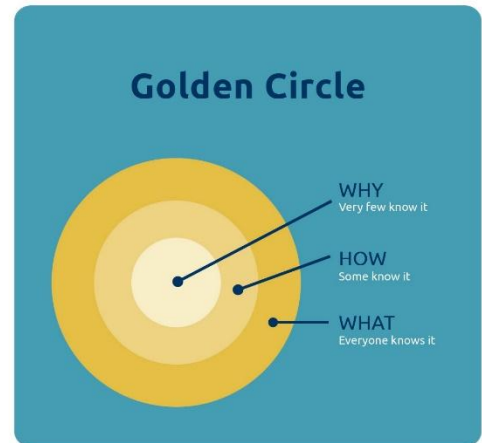
코드 리뷰, 왜 해야 할까요?

- 버그의 조기 발견
- 개발 표준(convention) 준수
- 중복 코드 방지 & 모듈의 재사용성 증대

출처: 코드 컴플릿(Code Complete), 스티브 맥코넬(Steve McConnell)

- 코드의 정확성 및 품질향상
- 당면한 문제에 대해 다양한 관점에서 개선 및 해결책 제시
- 새로운 아이디어에 대한 의견공유
- 사내 Code Convention 정착
- 코드 작성자가 놓친 오류 방지
- 팀의 협업 능력을 높이기 위한 노력
- 조직의 개선점을 찾고 같이 해결하는 선순환의 시작

출처: Code review one page paper Code Review in Agile Teams — part II, Wojciech Seliga)



코드 리뷰가 어려운 이유

Diversity의 이해 & Ground Rule의 수립

[NOTE]
상호 교류의 과정 중 기억하고 싶은 내용을 메모해 보시기 바랍니다.

Social Style 진단

1	자신을 주위에 맞춘다	D	C	B	A	자신이 주도권을 쥔다
2	조용하다	D	C	B	A	말하기를 좋아한다
3	추종적이다	D	C	B	A	지배적이다
4	질문한다	D	C	B	A	단언한다
5	협력적이다	D	C	B	A	경쟁적이다
6	내성적이다	D	C	B	A	외향적이다
7	여유 있는 페이스로 말한다	D	C	B	A	빠른 페이스로 말한다
8	자기 주장을 하지 않는다	D	C	B	A	자기 주장을 한다
9	자신을 억제한다	D	C	B	A	개방적이다

☐ 각 항목별 체크된 개수를 기입합니다.

D

C

B

A

10	냉철하다	1	2	3	4	쉽게 흥분한다
11	침착하다 안정적이다	1	2	3	4	활기 넘친다
12	업무 중심적이다	1	2	3	4	관계(인간) 중심적이다
13	엄격하고 예리하다	1	2	3	4	따뜻하고 사려 깊다
14	업무 중심으로 대화한다	1	2	3	4	자신의 감정을 이야기한다
15	차분하다	1	2	3	4	활달하다
16	사실을 중시한다	1	2	3	4	의견을 중시한다
17	감정에 좌우되지 않는다	1	2	3	4	감정에 좌우된다
18	감정을 드러내지 않는다	1	2	3	4	감정을 드러낸다

☐ 각 항목별 체크된 개수를 기입합니다.

1

2

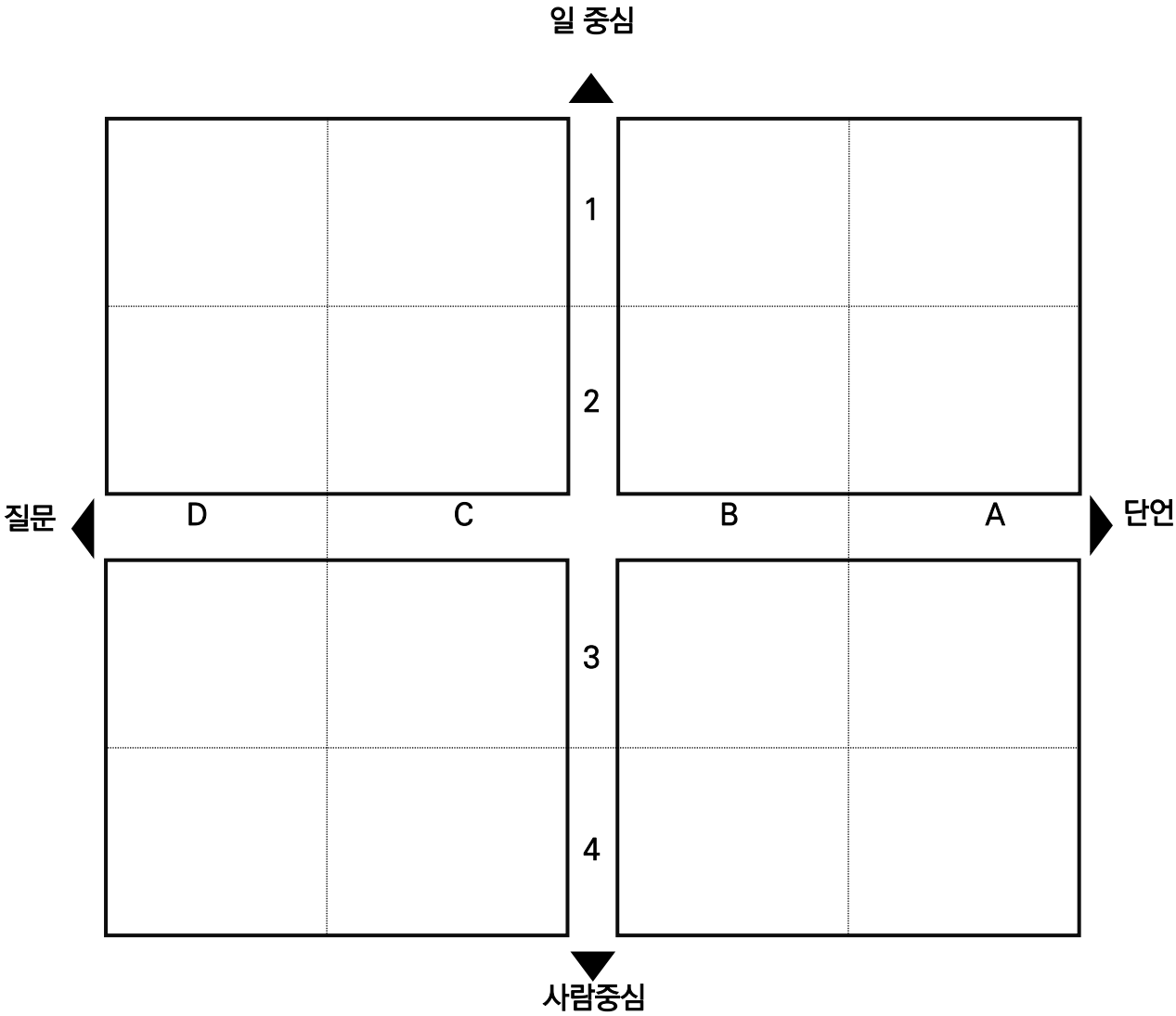
3

4

Social Style 진단

앞의 진단지에 선택한 알파벳 또는 숫자의 개수를 아래의 표에 적어 정리합니다.

D	C	B	A
1	2	3	4



코드 리뷰를 위한 환경 만들기



“300줄에서 400줄 정도의 코드를
60분에서 90분 동안 리뷰하면 70~90%의 결함을 발견할 수 있다!”
- by Cisco 시스템 프로그래밍 팀의 연구

MEMO

바람직한 코드 리뷰를 위한 소통 레시피 1: 효과적인 소통을 이끌어 내는 SBI 피드백

연구에 따르면, 개발자들은 긍정적인 피드백보다 수정 피드백을 선호하는 경향이 더 높고, 반면 리뷰어는 수정 피드백을 제공하기를 꺼린다고 한다. 의도가 정확히 전달될 지 여부를 확신할 수 없기 때문이다. 하지만 적절한 때의 피드백을 대충 넘기는 것은 팀/부서의 성장에 큰 문제 요인이 된다. 문제 코드의 개선이 이루어지지 않아 누적된다면 팀/부서의 성과에 저해가 되기 때문이다.

수정과 개선을 위한 피드백을 하고 싶지만 어떻게 입을 떼야 할 지 고민스러운 리뷰어를 위해 관계가 나빠지지 않으면서도 성과 개선을 기대할 수 있는 SBI 피드백 모델을 소개한다. SBI 피드백 모델이란 Center for Creative Leadership에서 개발한 피드백 도구로, 효과적인 피드백을 제공하는 데 사용할 수 있는 간단한 구조와 기법으로 구성된 대화 모델이다. SBI의 구조와 단계 별 대화 전략은 다음과 같다.

S: Situation – 상황을 포착한다.

명확하고 구체적으로 상황을 설명한다. "전날" 또는 "지난 주 회의에서"와 같은 모호한 용어는 사용하지 않는다.

예) "어제 아침의 프로젝트 회의에서 프레젠테이션을 했을 때 ..."

B: Behavior – 행동을 설명한다.

해결하고자 하는 정확한 행동에 대해 논의한다. 가정이나 주관적인 판단을 내리지 않는다. 다른 사람의 의견도 반영하지 말고, 부서장이 관찰한 내용만 이야기한다. 비난처럼 들리지 않도록 유의한다.

예) "두 개의 슬라이드에 대한 질문에 답할 수 없다는 사실을 알게 되었습니다. 시연 시 OO코드의 계산도 잘못되었습니다."

I: Impact – 마지막으로 그 사람의 행동이 개인, 팀 및 조직에 미치는 영향을 강조한다.

부서장의 주관적인 의견이 담긴 진술을 한다. "나" 또는 "우리"를 사용하여 요점을 만든다.

예) "전체 이사회가 거기에 있었는데, 그 것은 우리 팀의 평판에 영향을 미쳤습니다."

위의 요소를 합치면 다음과 같이 하나의 피드백이 완성된다.

"(S)어제 오전 팀 회의에서 프레젠테이션을 했을 때 (B)두 개의 슬라이드에 대한 질문에 답할 수 없다는 사실을 알게 되었습니다. 시연 시 코드 계산도 잘못되었습니다. (I) 전체 이사회가 거기에 있었는데 우리 팀의 평판에 영향을 미쳤습니다."

피드백을 해야 하는 이유는, 팀원 및 개발자가 스스로 깨우치는 발전에 도움이 되기 때문이다. 위에 제시한 SBI 피드백과 같이, 객관적인 관점에서 감정이나 감이 아닌 주어진 정보와 결과 지표를 기준으로 현 상황을 짚고 개선할 수 있는 방법에 대해 논의한다면, 개발자는 피드백을 바탕으로 개발 방식을 돌아보고 더 잘 할 수 있는 방법을 강구하기 위해 노력할 것이다.

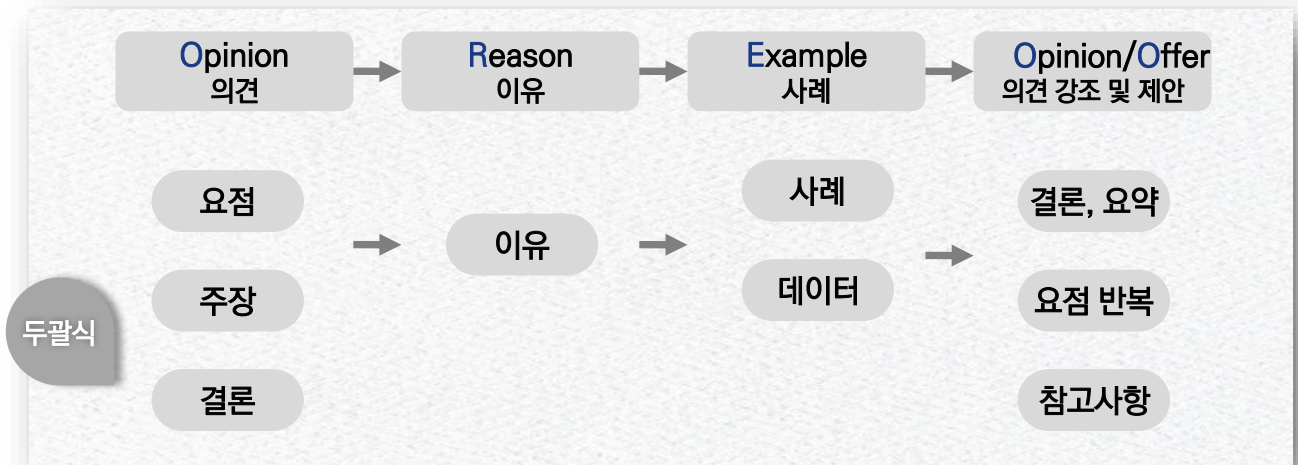
개발자의 코드 리뷰에서 피드백의 역할

코드리뷰(Code Review)는 개발자가 작성한 코드를 다른 개발자가 검토하고 피드백하는 과정으로, 코드의 품질을 높이고 버그를 사전에 방지하며 개발팀의 코드 스타일을 통일하는 데 중요한 역할을 합니다. 단순한 코드 검토가 아니라 개발자의 성장과 팀워크 향상에도 기여하는 필수적인 과정입니다. 즉, 코드 리뷰는 소프트웨어 개발 프로세스에서 중요한 부분을 차지하며, 팀의 코드 품질을 향상시키고 개발자들의 기술을 성장시키는 데 기여합니다.

코드 리뷰 과정에서의 피드백은 다양하고 중요한 역할을 수행합니다. 각 역할을 자세히 살펴보면 다음과 같습니다.

- **코드 품질 향상:** 피드백은 코드의 가독성을 높이고, 오류를 줄이며, 유지보수성을 향상시키는 데 도움을 줍니다. 리뷰어는 코드의 논리적 흐름, 명명 규칙, 주석의 적절성 등을 점검하여 코드가 일관되고 이해하기 쉽게 작성되었는지 확인합니다. 이를 통해 코드의 전반적인 품질이 향상됩니다.
- **버그 발견 및 예방:** 코드 리뷰 과정에서 피드백을 통해 미처 발견하지 못한 버그를 찾을 수 있습니다. 코드 구현의 논리적 결함이나 예외 처리의 누락 등을 리뷰어가 발견하여 피드백을 제공하면, 이러한 문제를 사전에 해결할 수 있습니다. 이는 제품의 신뢰성을 높이는 데 중요한 역할을 합니다.
- **개발자 학습과 성장:** 피드백은 개발자에게 학습 기회를 제공합니다. 경험이 많은 리뷰어로부터의 피드백은 코드 작성 방법, 새로운 기술, 모범 사례 등에 대해 배울 수 있는 기회를 제공합니다. 이를 통해 개발자는 자신의 기술을 향상시키고 더 나은 코드를 작성할 수 있게 됩니다.
- **팀의 일관성 유지:** 피드백은 팀 내에서 코드 작성 스타일과 규칙을 일관되게 유지하는 데 기여합니다. 리뷰어는 팀의 코딩 표준을 준수하도록 피드백을 제공하여, 코드베이스가 일관성을 유지할 수 있도록 합니다. 일관된 코드베이스는 협업을 원활하게 하고 코드 유지보수를 용이하게 합니다.
- **협업과 커뮤니케이션 강화:** 코드 리뷰는 팀원 간의 협업과 커뮤니케이션을 강화하는 기회를 제공합니다. 피드백을 주고받는 과정에서 팀원들은 서로의 아이디어를 공유하고, 문제를 해결하기 위한 최선의 방법을 논의할 수 있습니다. 이러한 상호작용은 팀의 결속력을 높이고, 공동의 목표 달성에 기여합니다.
- **총체적으로,** 코드 리뷰에서의 피드백은 단순히 코드를 수정하는 것을 넘어, 개발자의 역량 강화와 팀의 성공적인 협업을 위한 중요한 도구로 작용합니다. 이를 통해 소프트웨어의 품질과 효율성이 지속적으로 향상될 수 있습니다.

바람직한 코드 리뷰를 위한 소통 레시피 2: 효과적인 message 전달을 위한 Yes and 화법의 활용



Opinion

- 주어진 질문의 답을 말함, 자신의 의견을 말함

Reason

- 주어진 시간에 따라 적절한 분량의 근거 제시

Example

- 가급적 생생하고 구체적이며 친근한 사례
뒷받침하는 사례나 증거 제시

Opinion /Offer

- 핵심 강조 & 개선 사항 및 향후 실행 방안 제시

바람직한 코드 리뷰를 위한 소통 레시피 3: 효과적인 소통을 이끌어 내는 Yes, And 화법

No, Because	vs	Yes, And
일단 상대방의 말을 부정부터 한다.		우선, 상대방 의견이 일리가 있다고 인정해준다.
아니, 그게 아니구요! 왜냐하면~ 근데 저라면~		맞아요, 그렇구나! 그럴 수 있어요! 그리고 제 생각에는~
상대방의 의견에 먼저 반대하고, 그 이유를 밝힌다.		상대방의 의견을 먼저 인정하고, 나중에 자신의 의견을 밝힌다.

코드리뷰 팀커뮤니케이션 강화를 위한 ERRC Action Plan

ERRC 프레임워크를 이용한 액션플랜		
구분	구체적인 실행 계획	실행 여부 확인 방법 (측정 가능 지표)
E Eliminate	하지 말아야 행동 또는 태도	
R Reduce	줄여야 할 행동 또는 태도	
R Raise	증가시켜야 할 행동 또는 태도	
C Create	새롭게 시도해 볼 행동 또는 태도	
실행 과정에서 발생 가능한 예상 장애 요인		장애요인 해결방법